

TAREA 3 SIMULADOR GRÁFICO EN C++ Y QT DE ELOTELTags Y APLICACIÓN FIND MY

NOMBRES: JULIÁN NÚÑEZ, VICENTE MUJICA, VALENTINA SOTO Y VICENTE ANDRADE
PROFESOR: AGUSTÍN GONZÁLEZ VALENZUELA

Introducción

Esta tarea consiste en un simulador gráfico, desarrollado en C++ con la biblioteca Qt, que modela un sistema de localización de objetos inspirado en la tecnología AirTag de Apple, denominado EloTelTag. El sistema simula la interacción entre tres tipos de equipos (EloTelTags, celulares y tablets) que se desplazan sobre un territorio, manteniéndose confinados a él y rebotando en sus bordes. El objetivo es reimplementar en C++ y Qt el sistema previamente desarrollado en Java y JavaFX, conservando sus funcionalidades pero adaptándolo a un nuevo lenguaje y entorno gráfico.

El desarrollo se realizó de forma incremental en cuatro etapas. Las primeras abordan la lectura del archivo de configuración y el despliegue de los equipos en pantalla; las últimas incorporan el reporte de posiciones a una nube de datos y la aplicación Find My, que permite consultar la ubicación de los bienes de cada persona. El presente informe documenta la etapa final (Stage 4), en la que se integran todas las funcionalidades del sistema.

Desarrollo de Stage 4

En la etapa final se consolidan todos los requerimientos del simulador. A continuación se describen las funcionalidades implementadas y cómo se relacionan las clases del sistema, el cual se organiza en tres capas: el **modelo**, que representa los equipos y la nube de datos; la **vista**, que dibuja dichos equipos sobre el territorio; y la **aplicación**, que coordina la simulación y la interacción con el usuario.

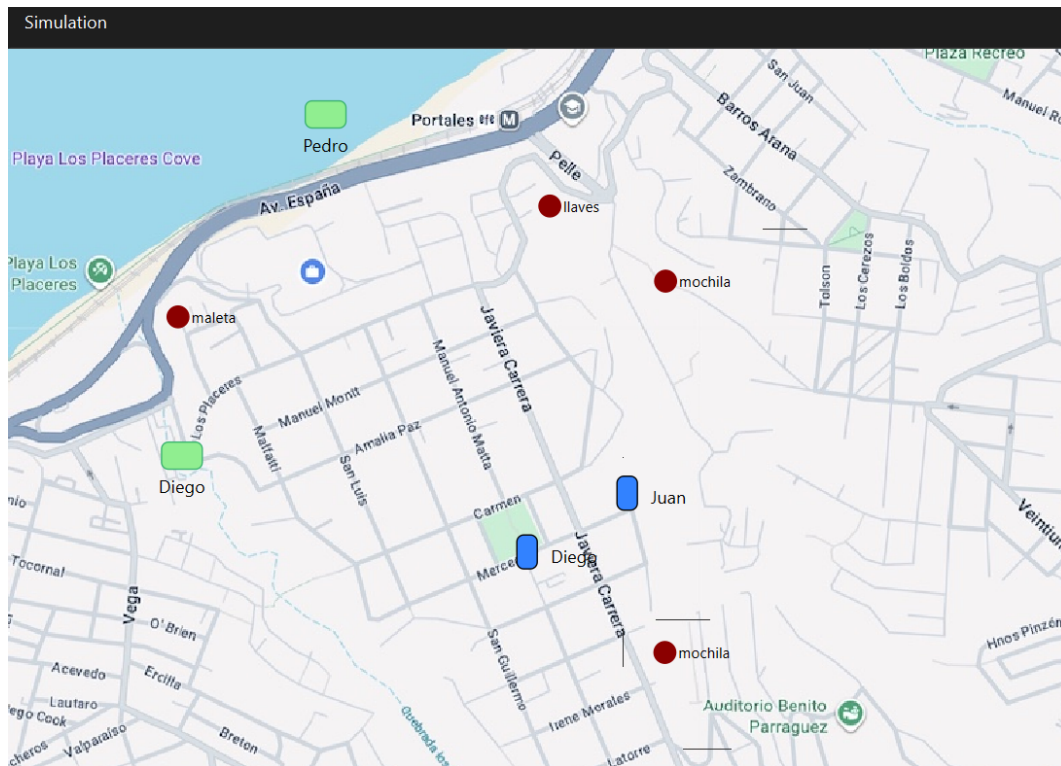


Figure 1: Simulación de Stage 4 con los equipos en movimiento.

Simulación y movimiento de los equipos

La simulación está gobernada por la clase principal de la aplicación, que avanza el tiempo en intervalos discretos y, en cada paso, actualiza la posición de todos los equipos. Estos comparten una clase base común (Equipo), de la que heredan los celulares, los EloTelTags y las tablets, todos capaces de moverse por el territorio y rebotar al alcanzar sus bordes. El usuario controla la simulación mediante un menú con las opciones Play y Pause, que inician y detienen el avance del tiempo sin perder el estado de los equipos.

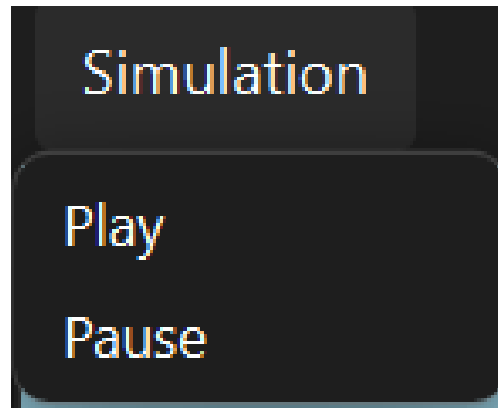


Figure 2: Opciones del menú Simulation: Play y Pause.

Reporte de posiciones y señal de radar

El sistema incorpora una nube de datos (ETNube) encargada de almacenar la información de localización. Los celulares reportan su propia posición de forma periódica, mientras que los EloTelTags y las tablets buscan celulares cercanos emitiendo una señal de radar a intervalos regulares, sin sincronizarse entre sí. Esta búsqueda se representa visualmente como un círculo que se expande desde el equipo hasta un radio determinado. Cuando un celular queda dentro de ese radio, lo detecta y registra en la nube tanto su propia posición como la del equipo rastreado. De esta manera, la ubicación de un tag o tablet se actualiza únicamente cuando es detectado por un celular cercano, reproduciendo el funcionamiento de un sistema de rastreo real, en el que la nube solo conoce el último lugar donde el objeto fue visto.

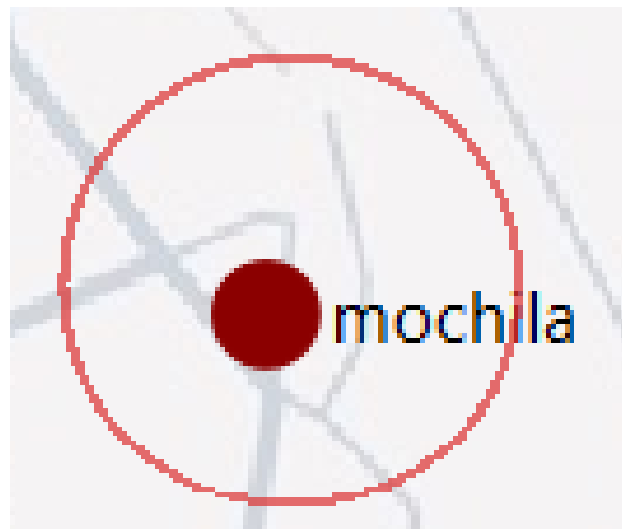


Figure 3: Señal de radar emitida por un equipo en busca de celulares cercanos.

Aplicación Find My y función traza

La funcionalidad Find My se activa al presionar sobre la vista de un celular o tablet, lo que despliega un menú emergente. Al seleccionarla, se abre una ventana que muestra la información de los equipos del dueño consultando la nube. A diferencia de la etapa anterior, donde esta ventana era estática, ahora se actualiza de manera continua mientras permanece abierta, reflejando en tiempo real el movimiento de los equipos. La simulación no se interrumpe mientras la ventana está activa.

Asociado a cada equipo listado en esta ventana se incluye además la función traza, que dibuja sobre el mapa la trayectoria seguida por el equipo desde el momento en que se activa. Al desactivarla, la trayectoria desaparece. Esta función reemplaza a los efectos de sonido presentes en la versión anterior del sistema.

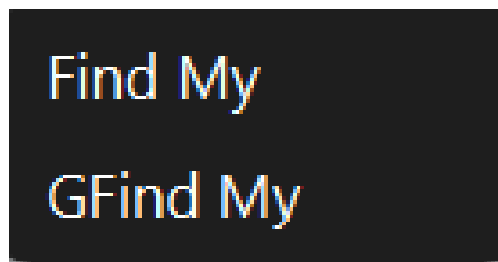


Figure 4: Menú emergente al presionar sobre un celular o tablet.

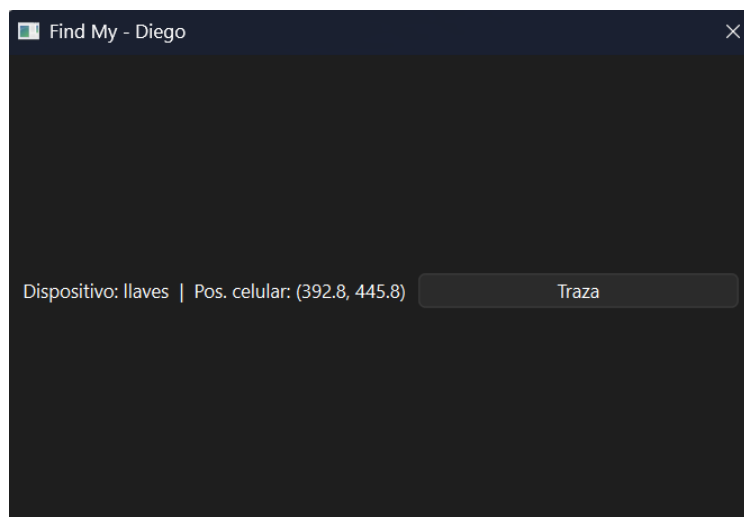


Figure 5: Ventana Find My con la información de los equipos del dueño.

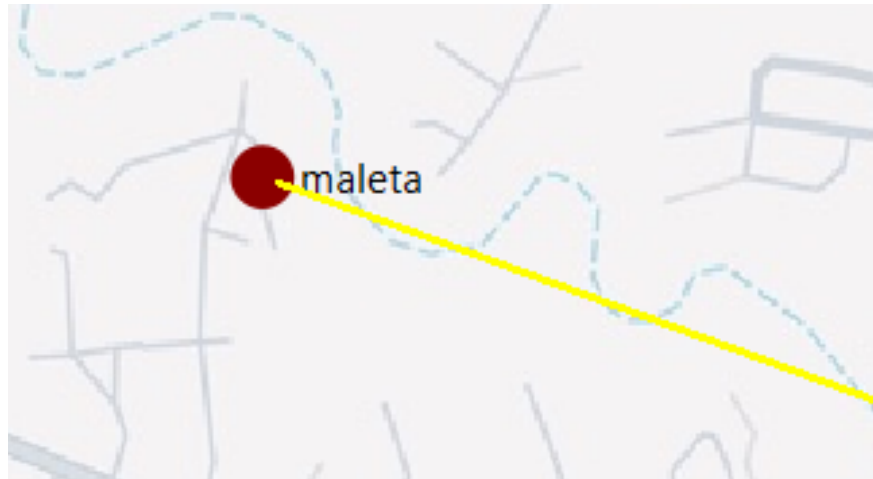


Figure 6: Función traza: trayectoria de un equipo dibujada sobre el mapa.

Visualización geográfica GFind My (extra)

Como funcionalidad adicional, el menú emergente incluye la opción GFind My (Graphics Find My). Al seleccionarla se abre una ventana que reproduce la imagen del territorio y dibuja sobre ella los equipos del dueño, cada uno representado según su tipo. Esta ventana también se actualiza periódicamente con la información de la nube, por lo que el celular, al reportar su posición de forma continua, se observa en movimiento, mientras que los tags y tablets permanecen en su última posición detectada hasta que un celular vuelve a pasar cerca de ellos.

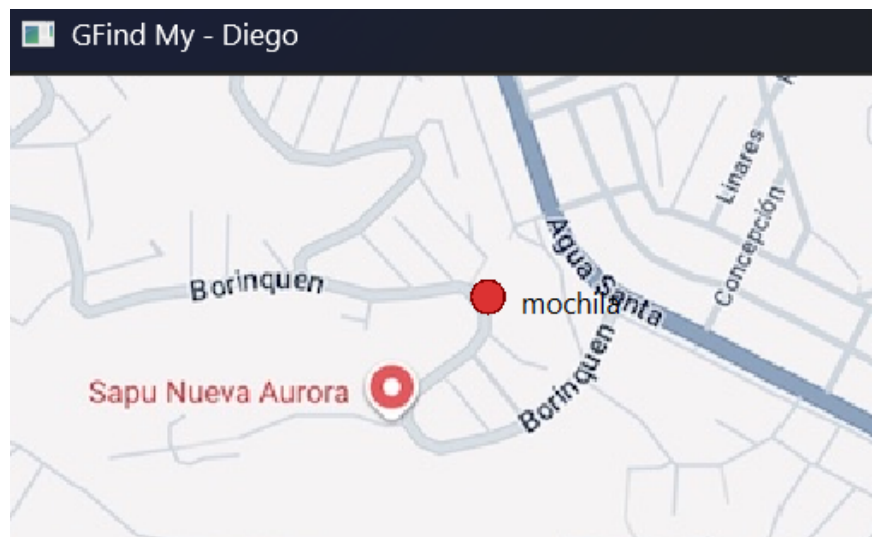


Figure 7: Ventana GFind My: representación geográfica de los equipos del dueño.

Conclusión

El desarrollo de esta tarea permitió reimplementar exitosamente en C++ y Qt un sistema previamente construido en Java y JavaFX, cumpliendo con todos los requerimientos planteados e incorporando además la funcionalidad extra GFind My. El trabajo por etapas incrementales facilitó la construcción del sistema de forma ordenada, validando cada conjunto de funcionalidades antes de avanzar al siguiente.

La principal diferencia respecto a la versión anterior radicó en adaptarse al modelo gráfico y de eventos propio de Qt, distinto al de JavaFX, lo que exigió comprender cómo se gestionan los temporizadores y la interacción con los elementos en pantalla. La separación del sistema en las capas de modelo, vista y aplicación resultó fundamental para mantener el código ordenado y permitir que las distintas funcionalidades se integraran sin afectar al resto del sistema.

Diagrama de Clases (UML)

El siguiente diagrama representa la arquitectura de la aplicación en su etapa final, organizada en las capas de modelo, vista y aplicación descritas a lo largo del informe.

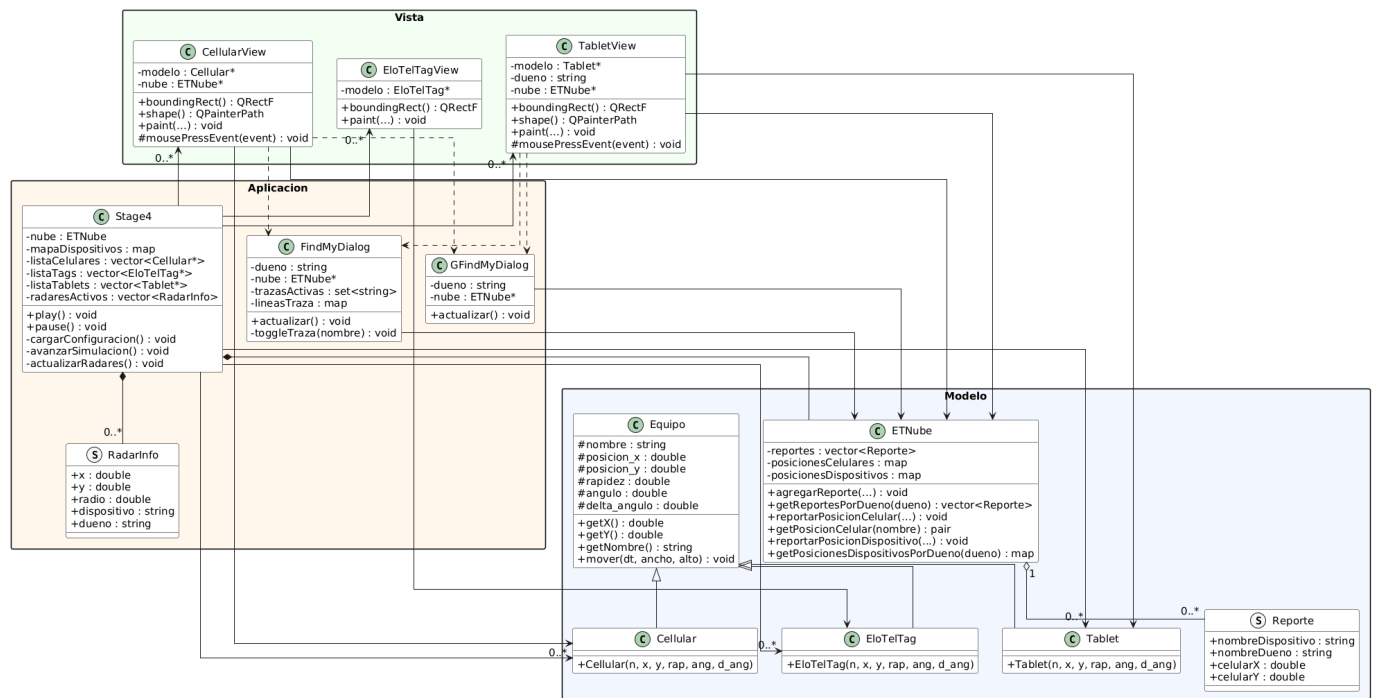


Figure 8: Diagrama de clases UML de la aplicación (Stage 4).